

Reading _s2_closer

Recovering unmarked spans by bracket-matching — and why that is bounded Dyck

`experiments/22_rasp_l_hi_hf/_s2_closer.py` (Strategy A, the chosen winner)

This program turns a *head-initial* sentence into its *head-final* reordering using only the length-generalizing transformer primitives of RASP-L (Weiss et al. 2021; Zhou et al. 2024): elementwise maps, relative-position reads, and attention-based counting. The whole task reduces to one hard sub-problem — **recovering the constituent spans from a flat string with no brackets** — and `_s2_closer` solves it by explicitly finding, for each clause opener, where its clause *closes*. That is bracket-matching, and its cost in layers is exactly the story of **bounded Dyck** languages.

1 The pipeline and where closer sits

HF is a *permutation* of HI: every token keeps its identity and only moves. So the program never “generates” — it computes, per HI position i , a target slot and gathers:

$$\text{hf_pos}(i) = i + \text{disp}(i), \quad \text{disp}(i) = \sum_{F \text{ flip}, i \in F} (+|\text{comp}(F)| \text{ (} i \in \text{head} \text{)}, -|\text{head}(F)| \text{ (} i \in \text{comp} \text{)}).$$

Three stages:

stage	what	primitives
1 local POS + role	word $\rightarrow$ lexical class $\rightarrow$ fine POS & verb role	<code>tok_map</code> + relative reads
2 spans (closer)	recover clause/phrase <i>end</i> of every opener, then depth & disp	relaxation + <i>stabbing</i> counts
3 gather	invert the permutation, place tokens	two <code>kqv</code> heads

Stages 1 and 3 are easy and provably length-generalizing (Stage 3 was verified 2000/2000 when fed the oracle’s `disp`, independent of Stage 2). **Everything hard is Stage 2**, and `_s2_closer` is one of four independent constructions of it. This handout reads that construction end-to-end.

2 Why spans are the hard part: unmarked, coincident closes

To know which flip nodes enclose token i — and how big their complements are — we need each constituent’s **span** $[q, e(q)]$. In a bracketed string that is trivial. Here it is not:

Two facts that make this genuine bracket-matching

- (1) **Closes are unmarked.** The surface string has openers you can *see* (a **that** begins a clause) but *no* close symbol — a clause just ends where its verb phrase ends, which you must *infer* from parts of speech.
- (2) **Closes can coincide (multiplicity > 1).** A `CP_sent` sitting at the right edge of a relative clause closes *together* with it: in *the cat [that claims [that the dog swims]] runs*, both clauses end right after *swims*.

So Stage 2 must *compute* the matching close $e(q)$ for every opener q . The recursion is real: a clause’s end depends on its complement’s end, which for an embedded clause is *another* clause’s end. Once $e(q)$ is known, the nesting depth is a count:

$$\text{depth}(i) = \#\{\text{openers } q \leq i : e(q) \geq i\}$$

(how many opener–close intervals “stab” position i).

3 Stage 1: lexical class and verb role (all local)

A fixed word→class embedding, then four relative-position reads to see the neighbours, then a few elementwise decisions. No cross-clause information yet.

```
lc      = tok_map(hi_ids, _lexclass)          # D, N_sing, N_prop, Adj, Adv, THAT, V, OTHER
prev_lc = _read_off(lc, -1); next_lc = _read_off(lc, +1) # lc[i-1], lc[i+1] (rel. reads)
is_crel = isthat & (prev_lc == NSING)        # 'that' after a common noun -> relative C_rel
nn_lc   = where(next_lc == ADV, next2_lc, next_lc) # next NON-adverb class
role    = where(nn_lc == THAT, 3,             # V_cp (takes a clause)      role 3
               where((nn_lc==D)|(nn_lc==NPROP), 2, # V_dp (takes an object DP)  role 2
               where(is_intr, 1, 4)))          # intransitive 1 / object-gap 4
```

Verb role is the key local disambiguation: a verb's role decides whether it heads a **flip** VP and what its complement is. Roles: 1=intransitive, 2=takes an object DP, 3=takes a clause, 4=object-gap (a transitive verb whose object was relativized away, e.g. *that the boy chases*). Roles 2 and 3 are the VP-flip heads; 1 and 4 never flip. All of Stage 1 is content + relative offset ⇒ length-generalizing.

4 Stage 2a: the **closer** — END pointers by relaxation

The core. We maintain four **end pointers**, one per constituent type, initialised to “ends at itself” and then refined by a fixed number K of *relaxation sweeps*:

pointer	meaning: last index of the ... starting here
dpfull	DP (<i>including</i> any relative clause hanging off its noun)
vpe	VP (verb + its complement: object DP, or embedded CP, or nothing)
rce	CP_rel (a relative clause <i>that</i> ...)
cpe	CP_sent (a sentential clause <i>that</i> ...)

```
dpfull, vpe, rce, cpe = dp_end_flat, hb_end, idx, idx # init: each "ends at itself"
for _ in range(K): # each sweep resolves ONE more nesting level
    dpfull = where(dp_has_rc, rce_at_rc, dp_end_flat) # DP skips its rel. clause
    vpe = where(role in {1,4}, hb_end,                # intrans/objgap: stop at the verb
               where(role==2, dpfull_at_hb1,          # +object DP: end of that object
               where(role==3, cpe_at_hb1, idx)))      # +clause: end of the that-CP
    rce = where(is_crel, subj?vpe_at_q1:dpfull_at_q1+1, idx) # CP_rel end
    cpe = where(is_c, vpe_at_vpos, idx)                  # CP_sent: its inner VP's end
    e = where(is_crel, rce, where(is_c, cpe, idx))      # clause end e(q) per 'that'
```

Why a *sweep* is one attention hop

Each assignment reads a pointer *at another position* (`index_select`, one attention head): e.g. a clausal verb's `vpe` copies the `cpe` of the *that* just after it; that `cpe` in turn copied the `vpe` of *its* inner verb. A single sweep can only follow **one** such link. To resolve a clause whose complement is a clause whose complement is a clause, you must run the sweep once per level: **sweeps needed = nesting depth**. This is the load-bearing fact for everything in §8.

5 Stage 2b: depth and displacement as *stabbing sums*

With $e(q)$ known, **depth** is a single counting head — count opener intervals that cover i :

```
end_that = where(isthat, e, -1) # only 'that' positions carry a real interval end
depth    = sel_width(select(end_that, indices, geq, causal=True)) # #{q<=i : e(q) >= i}
```

The *same* stabbing trick expresses the whole displacement. Grouping the per-flip sum by **flip type** gives four additive pieces, each a (weighted) count of enclosing flips of that type plus a local head bonus:

piece	flip type	how it is computed
d_adv	VP_x_adv (verb↔adverb)	purely local: verb +1 if adverb follows, adverb −1
d_cp	CP_sent + CP_rel	−depth everywhere; each that also + its clause
d_nprel	NP_singular→[core, CP_rel]	core noun + RC ; RC tokens − core per enclosing RC (stab)
d_vp	transitive/clausal VP	head-block + comp ; comp tokens − head per enclosing VP (stab)

The one subtlety in reading these pieces

Each of d_cp, d_nprel, d_vp is a **stabbing sum**: a token pays into *every* enclosing flip of that type, not just its parent. That is why, below, *likes* gets d_vp = +1: it is the **head** of its own object-taking VP (+2) *and* sits inside the outer clause-taking VP’s complement (−1). Same additive rule as the tree formula in §1, just bucketed by flip type. All four sum to the oracle’s **disp** (verified, and still exact at depth 3 — it is a tree identity, not a depth-2 coincidence).

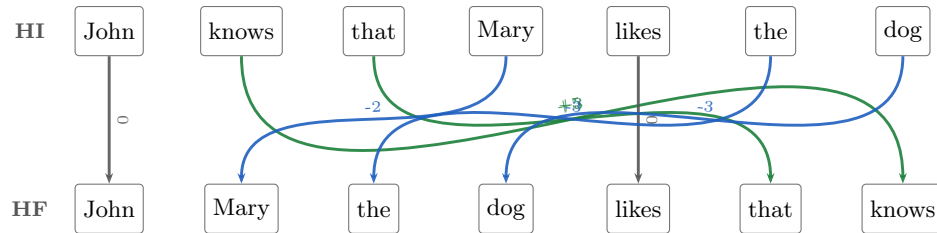
Stage 3 then inverts the permutation and gathers (two heads):

```
hf_pos = idx + (d_adv + d_cp + d_nprel + d_vp)
src     = kqv(hf_pos, idx, idx, equals, causal=False)  # slot j <- the i with hf_pos[i]==j
out     = index_select(hi_ids, src, causal=False)       # place each HI token into its slot
```

6 Worked example (depth 1), every number verified

John knows that Mary likes the dog → *John Mary the dog likes that knows*. One opener, **that** at position 2, closing at $e = 6$ (the clause *that Mary likes the dog* ends on *dog*). Roles: *knows*=clausal (3), *likes*=transitive (2).

i	tok	role	$e(q)$	depth	d_adv	d_cp	d_nprel	d_vp	disp
0	John	—	—	0	0	0	0	0	0
1	knows	cp	—	0	0	0	0	+5	+5
2	that	—	6	1	0	+4	0	−1	+3
3	Mary	—	—	1	0	−1	0	−1	−2
4	likes	trans	—	1	0	−1	0	+1	0
5	the	—	—	1	0	−1	0	−2	−3
6	dog	—	—	1	0	−1	0	−2	−3



Read d_cp: everyone in the clause body gets $-\text{depth} = -1$; the **that** additionally gets $+|\text{clause}| -$ (outer CPs) $= +4$, i.e. it is the clause’s head jumping over its 4-token body. d_vp: *knows* is a clausal VP

9 Theory implications

9.1 The demonstrated law: layers = depth + 1

ROUNDS is a fixed block of transformer layers. Sweeping it reproduces a sharp wall — re-verified directly: ROUNDS=4 solves depth 3 (40/40), while ROUNDS=3 fails it entirely (0/40):

$$R \text{ rounds resolve CP-depth } \leq R - 1; \quad \text{depth } R \text{ fails entirely.}$$

Because a clause’s end depends on its complement’s end (another clause, for `V_cp`), the END-pointer dependency chain is as long as the nesting is deep; one sweep extends every pointer by one level, so resolving depth d needs $\Omega(d)$ *sequential* sweeps = $\Omega(d)$ layers. A **fixed**-depth transformer therefore represents HI→HF only up to some depth and cannot extend to deeper inputs — **no amount of data helps**, because deeper inputs need more sequential composition than the architecture physically has.

9.2 Length generalization $\neq$ depth generalization

This is the crucial distinction. *At a fixed depth*, every op in the program is content-based, relative-position, or counting — the RASP-L primitives that **do** length-generalize (Zhou et al. 2024): a depth-2 sentence that is merely *longer* is handled fine. What does *not* generalize is **depth**: the only programs for the matching function use unbounded-depth bracket-matching, which lives outside the length-generalizing primitive set. So structured reversal joins copy / reverse / unbounded-Dyck as a task that “does not length-generalize without a hint,” and the ROUNDS knob makes the boundary explicit and testable.

9.3 Where this sits in expressivity theory

- **Bounded Dyck is doable, at a price.** Transformers *can* recognize Dyck- (k, D) , but the required depth grows with D (Yao et al. 2021). Our linear **layers=depth+1** is a concrete, mechanism-level instance of that theorem.
- **Unbounded Dyck is out of reach.** Fixed-depth (log-precision) transformers sit in a shallow, uniform-TC⁰-style circuit class (Merrill & Sabharwal 2023); an unbounded stack for arbitrary nesting is outside it, and hard-attention cannot do unbounded Dyck or PARITY at all (Hahn 2020). Depth generalization to *arbitrary* nesting is thus a hard *expressivity* barrier, not a mere optimization one.

9.4 Falsifiable predictions for the training splits

Two axes that must behave oppositely

Depth held out (train depth ≤ 1 , test depth 2): the model is never forced to *use* the second sweep, so at depth 2 it hits the wall $\Rightarrow$ **predict failure** (matches the empirical negatives in exps 10/16).

Object-gap $\rightarrow$ subject-gap relatives (same depth): both gap types are the *same* CP_rel flip and need *no* extra nesting — the gap only changes what sits *inside* the clause $\Rightarrow$ **predict transfer succeeds**. A failure here would be optimization, not expressivity.

9.5 Escape hatches (both consistent with the theory)

(a) **Give depth as an input hint** — index/scratchpad tricks (Zhou et al. 2024) so the model *reads* nesting instead of *deriving* it; this literally provides the non-length-generalizing part. (b) **Let layers scale with input** — universal / looped transformers restore the $\Omega(\text{depth})$ sequential budget, and depth generalization is back on the table. A plain fixed-depth decoder trained on shallow data gets neither, so the wall stands.

10 Why the wall is intrinsic, not a quirk of this code

The same $\Omega(\text{depth})$ wall appears in three independent Stage-2 constructions — `closer` (relaxation), `interval` (that/verb balance + pointer-jumps), `twolevel` (four families + fix-point) — each verified 100% at depth ≤ 2 and each failing one level past its round budget. A fourth (`gaplca`, pairwise flip-LCA) already breaks at depth 3. Three different algorithms hitting the *same* wall is the signature of a property of the *task*, not of a particular program: HI $\rightarrow$ HF requires bounded-depth bracket matching, and its layer cost is set by the depth D , exactly as Dyck- (k, D) theory predicts.

Every traced value (roles, $e(q)$ per sweep, `depth`, the four `disp` pieces, HF) was produced by `_trace_closer.py`, which reproduces `_s2_closer.hi_to_hf` verbatim and asserts agreement with it and with `grammar_oracle.py`.

References: Weiss, Goldberg & Yahav 2021 (RASP); Zhou et al. 2024 (RASP-L / length generalization); Yao et al. 2021 (bounded hierarchical languages); Hahn 2020 (self-attention limits); Merrill & Sabharwal 2023 (log-precision transformers $\subseteq \text{TC}^0$).